UNIVERSIDAD DE PAMPLONA

Formando líderes para la construcción de un nuevo país en paz

VARIABLES, LISTAS Y ARREGLOS: PYTHON Y RUST

Lezly Ximena Martinez Rubiano

Física computacional I
2026-1

Python

Asignación de Variables:

en Python no es necesario declarar el tipo de dato, pero es necesario conocer los diferentes tipos.

Tipos de Datos:

```
entero = 10        # int
decimal = 3.14     # float
texto = "Hola"     # str
activo = True      # bool
```

Restricciones: los nombres de variables solo pueden tener

letras

Números

Guion bajo

Rust

Asignación de Variables:

En rust el lenguaje es mas estricto por lo tanto para declarar variables utilizamos el comando “let”

por ejemplo

```
let edad = 15;
let nombre = "Carlos";
```

Variables inmutables: no cambian de valor fácilmente

```
let x = 5;
x = 10;  // ERROR
```

Variables mutables: Cambian de valor fácilmente

```
let mut x = 5;
x = 10;  //  sí funciona
```

Tipos de Datos:

```
let edad: i32 = 15; Enteros
let altura: f64 = 1.75; Decimales
let activo: bool = true; Booleanos
let letra: char = 'A’; Caracteres
```

Python

Listas: una **lista** es una estructura que permite **guardar varios valores en una sola variable**.

Características:

- Ordenadas
- Mutables(se pueden modificar)
- Permiten diferentes tipos de datos
  Usan corchetes []

Tipos de listas

- **Listas simples**: Contiene elementos del mismo tipo.
  números = [1, 2, 3, 4, 5]
- **Lista con diferentes tipos de datos**:
  datos = ["Juan", 20, 1.75, True]

Lista vacía: se usa cuando se van a agregar elementos después
lista = [] ó lista = []

Listas anidadas : listas dentro de listas, usa para representar: tables, matrices, estructuras bidimensionales

```
matriz = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
```

Python

Acceso a elementos: Cada elemento tiene un índice

```
numeros = [10, 20, 30]
print(numeros[0])
```

Modificar elementos:

```
numeros = [10, 20, 30]
numeros[1] = 50
```

Métodos principales de las listas:

- **Agregar elementos**

```
numeros.append(40)
```

- **Insertar posición:**

```
numeros.insert(1, 15)
```

- **Eliminar elementos:**

```
numeros.remove(20)
```

Arreglos en Python

Python **no tiene arrays nativos como otros lenguajes**, pero existen dos formas comunes.

Array con modulosarray:

```
import array
numeros = array.array('i', [1, 2, 3, 4])
```

Tipo: entero'i', float 'f' y double'd'

Arrays con NumPy: con librería de Numpy

```
import numpy as np
numeros = np.array([1, 2, 3, 4])
```

Rust

Arrays en Rust:
Un **array** es una colección de elementos **del mismo tipo y tamaño fijo**.
let arreglo: [tipo; tamaño] = [valores];
let numeros: [i32; 5] = [1, 2, 3, 4, 5];
Caracteristicas:
Tamaño fijo
Mismo tipo de dato
Memoria estatica

Acceso a elementos:
let numeros = [10, 20, 30];
println!("{}", numeros[0]);

Modificar Arrays: solo s el array es mutable
let mut numeros = [1, 2, 3];
numeros[0] = 10;

Vector (Vec):
El vector es como una lista de Python
tamaño dinámico
- Se puede modificar
- Se puede expandir

let mut numeros = vec![1, 2, 3];

Vector vacío:
let mut numeros: Vec<i32> = Vec::new();

Agregar valores
numeros.push(10);
numeros.push(20);

Slices: Un **slice** es una referencia a una parte de un arreglo o vector.
let numeros = [1, 2, 3, 4, 5];

let parte = &numeros[1..3];

Python

Condicionales (If): Python usa **indentación**.
Sirven para **ejecutar código dependiendo de una condición**.

```
edad = 18

if edad >= 18:
    print("Eres mayor de edad")
elif edad == 17:
    print("Casi eres mayor")
else:
    print("Eres menor de edad")
```

Rust

Condicionales: Usa llaves {}.

```
let edad = 18;

if edad >= 18 {
    println!("Eres mayor de edad");
} else if edad == 17 {
    println!("Casi eres mayor");
} else {
    println!("Eres menor de edad");
}
```

Python

Bucles (Loops): **repetir código varias veces**.

for

```
for i in range(5):
    print(i)
```

While

```
i = 0
while i < 5:
    print(i)
    i += 1
```

Loop infinito: Se usa cuando quieres repetir algo **hasta que ocurra una condición interna**.

```
while True:
    print("Hola")
```

break (detener el bucle)

Rust

Bucles: **repetir variables**

for

```
for i in 0..5 {
    println!("{}", i);
}
```

While

```
let mut i = 0;
while i < 5 {
    println!("{}", i);
    i += 1;
}
```

Loop infinito: Se usa cuando quieres repetir algo **hasta que ocurra una condición interna**.

```
loop {
    println!("Hola");
}
```

break (detener el bucle)